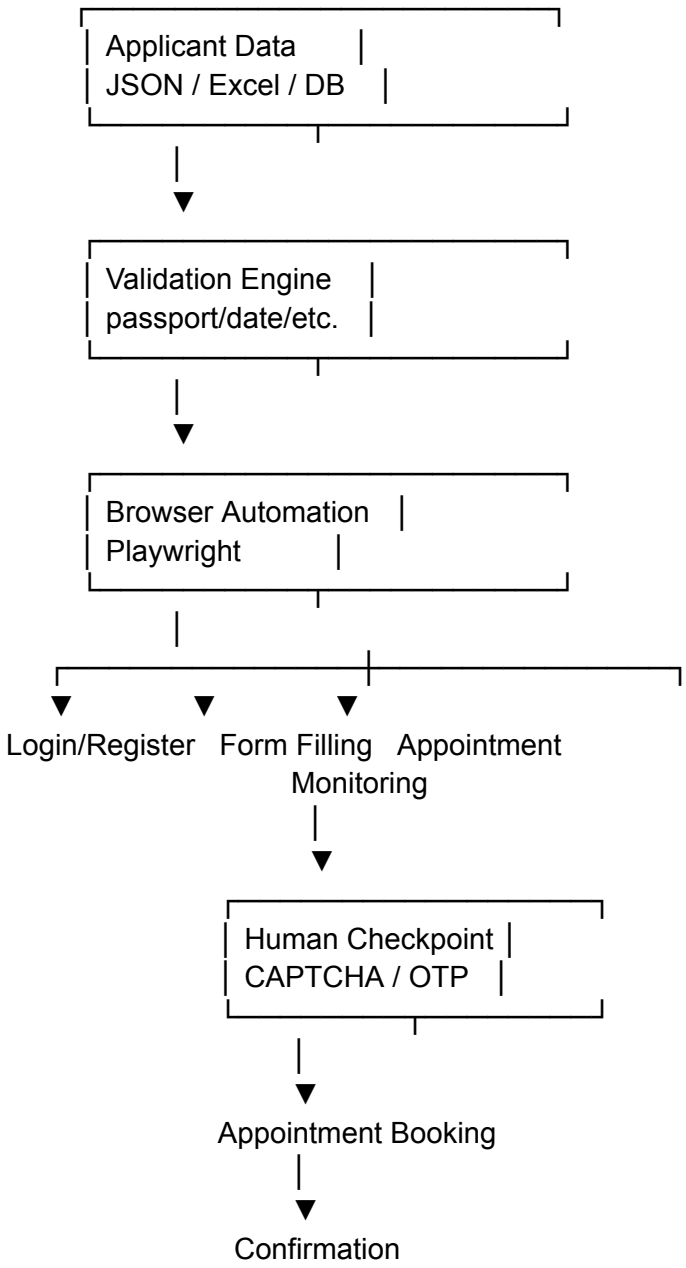
That distinction matters because VFS explicitly prohibits misuse/hacking/abuse of its website, and some VFS routes deliberately use CAPTCHA, identity verification, random slot releases, and other controls.

1. What I would build

A good architecture is:



For a new project, **Python + Playwright** would be my first choice.

---

2. Don't start with appointment booking

This is the biggest practical recommendation.

Build the system in stages:

Phase 1 — Form automation

Get this working first:

Applicant JSON
↓
Open VFS
↓
Login
↓
Navigate to application
↓
Fill applicant fields
↓
Validate fields
↓
Stop

Don't touch appointment hunting initially.

Phase 2 — Appointment-page automation

Then:

Open appointment page
↓
Select location
↓
Select visa/application category
↓
Read available dates
↓
Read available time slots
↓
Choose according to rules

Phase 3 — Human checkpoint

When VFS presents:

- CAPTCHA
- OTP
- identity verification
- suspicious activity challenge
- payment authentication

your automation should **pause**.

Example:

Automation running
↓
CAPTCHA detected
↓
PAUSE
↓
Browser remains open
↓
Human solves CAPTCHA
↓
Press "Continue"
↓
Automation resumes

This is considerably more reliable than trying to circumvent the protection.

---

3. Use Playwright rather than Selenium

I would recommend:

Python
+
Playwright
+
SQLite/PostgreSQL
+
JSON/Pydantic

+
optional **FastAPI**

Why Playwright?

It gives you:

- reliable browser control
- Chromium/Firefox/WebKit
- auto-waiting
- network monitoring
- screenshots
- tracing
- persistent browser contexts
- multiple pages/tabs
- easier debugging
- good handling of modern React/Angular websites

Basic structure:

```
vfs-automation/
│
├── app/
│   ├── browser.py
│   ├── login.py
│   ├── navigation.py
│   ├── applicant.py
│   ├── form.py
│   ├── appointment.py
│   ├── captcha.py
│   ├── payment.py
│   └── confirmation.py
│
├── models/
│   └── applicant.py
│
├── data/
│   └── applicants.json
│
├── tests/
│
├── screenshots/
│
├── logs/
│
```

```
├── config.yaml
│
└── main.py
```

---

4. Create a structured applicant database

Do not hard-code applicant information into your automation script.

Use a structured model.

For example:

```
{
  "applicant_id": "APP001",
  "first_name": "XXXX",
  "last_name": "XXXX",
  "date_of_birth": "1995-06-15",
  "passport_number": "XXXXXXXX",
  "passport_expiry": "2030-05-20",
  "nationality": "IND",
  "email": "example@example.com",
  "phone": "XXXXXXXXXX",
  "visa_type": "Tourist",
  "travel_date": "2026-12-10",
  "preferred_centres": [
    "Delhi",
    "Mumbai"
  ],
  "preferred_dates": {
    "from": "2026-10-01",
    "to": "2026-11-15"
  }
}
```

Obviously, don't put real passport numbers or credentials into source code or Git.

Use environment variables / encrypted storage for sensitive credentials.

---

5. Build a validation layer BEFORE VFS

This is extremely important.

VFS itself warns that appointment information needs to correspond to the applicant's passport/travel information, and some VFS processes make appointment fields non-editable after booking.

So your automation should catch mistakes **before** submitting anything.

For example:

```
def validate_applicant(applicant):
    assert applicant.first_name
    assert applicant.last_name
    assert applicant.passport_number
    assert applicant.date_of_birth
    assert applicant.passport_expiry

    if applicant.passport_expiry <= applicant.date_of_birth:
        raise ValueError("Invalid passport expiry")

    if not valid_email(applicant.email):
        raise ValueError("Invalid email")

    if not valid_phone(applicant.phone):
        raise ValueError("Invalid phone")
```

But don't stop there.

Create field-specific validators:

passport number
date of birth
passport expiry
nationality
email
phone
visa category
application category
centre
travel date

---

6. Never rely on coordinates

Avoid this:

page.mouse.click(734, 521)

It will break as soon as VFS changes the UI.

Instead use:

page.get_by_label("First Name").fill(first_name)

or robust selectors such as:

page.locator('input[name="firstName"]').fill(first_name)

or:

page.get_by_role("button", name="Continue").click()

The preferred selector hierarchy is roughly:

1. accessibility role/name
2. label
3. stable name/id
4. data-* attributes
5. CSS structure
6. XPath
7. screen coordinates ← avoid

---

7. Build a "page state detector"

This will make your automation much more robust.

Instead of assuming:

Step 1
Step 2
Step 3
Step 4

build a state machine.

For example:

class VFSState:

```
    LOGIN = "login"
    DASHBOARD = "dashboard"
    APPLICATION = "application"
    APPOINTMENT = "appointment"
    CAPTCHA = "captcha"
    OTP = "otp"
    PAYMENT = "payment"
    CONFIRMATION = "confirmation"
    ERROR = "error"
```

Then:

```
def detect_state(page):

    if is_login_page(page):
        return VFSState.LOGIN

    if is_dashboard(page):
        return VFSState.DASHBOARD

    if is_captcha(page):
        return VFSState.CAPTCHA

    if is_otp_page(page):
        return VFSState.OTP

    if is_payment_page(page):
        return VFSState.PAYMENT

    if is_confirmation(page):
        return VFSState.CONFIRMATION
```

This is much more resilient than a giant script containing hundreds of clicks.

---

8. Treat VFS as a dynamic application

This is particularly important.

Don't assume the HTML structure today will remain the same tomorrow.

Your automation should have:

Page detection
↓
Element detection
↓
Action
↓
Verification
↓
Next state

For example:

```
fill_first_name()

assert page.locator(
    'input[name="firstName"]'
).input_value() == applicant.first_name
```

After clicking Next:

```
page.get_by_role("button", name="Continue").click()

page.wait_for_load_state("networkidle")
```

Then verify the expected page exists.

---

9. Appointment selection should be rules-based

Don't simply say:

> Pick the first available appointment.

Instead create an appointment policy.

For example:

```
appointment:
  preferred_centres:
    - Delhi
    - Mumbai
```

```
  date:
    earliest: 2026-10-01
    latest: 2026-11-15

  preferred_weekdays:
    - Monday
    - Tuesday
    - Wednesday

  preferred_time:
    start: "09:00"
    end: "14:00"
```

Then your selector can calculate:

```
available slots
      ↓
remove wrong centre
      ↓
remove wrong visa category
      ↓
remove dates outside range
      ↓
remove undesirable times
      ↓
rank remaining slots
      ↓
select best valid slot
```

---

10. Don't create an aggressive polling bot

This is where many VFS automation projects become unreliable.

A naive implementation does:

```
while True:
    page.reload()
    check_slots()
    time.sleep(1)
```

I **wouldn't build it this way**.

VFS says some appointment slots are released at random times specifically to prevent manipulation.

A better architecture is:

Scheduler
  ↓
Open legitimate session
  ↓
Check appointment availability
  ↓
If unavailable:
    wait according to policy
  ↓
Check again

And introduce sensible limits:

max checks/hour
max session duration
max consecutive errors
max retries
cool-down after errors

If the site returns an explicit rate-limit/block response:

STOP
↓
record event
↓
wait
↓
require human review

Don't try to evade the block.

---

11. CAPTCHA should be a human checkpoint

This is one of the most important architectural decisions.

Don't design:

CAPTCHA
↓
automatically solve
↓
continue

Design:

CAPTCHA detected
↓
pause automation
↓
take screenshot
↓
show browser
↓
human completes challenge
↓
automation detects completion
↓
continue

For example:

```
if is_captcha(page):
    print("CAPTCHA detected.")
    print("Please complete CAPTCHA in the browser.")

    page.wait_for_function("""
        () => !document.querySelector('.captcha')
    """)
```

The exact detection mechanism will depend on the VFS implementation.

VFS itself describes CAPTCHA and human checks as part of measures used to block fraudulent appointment activity.

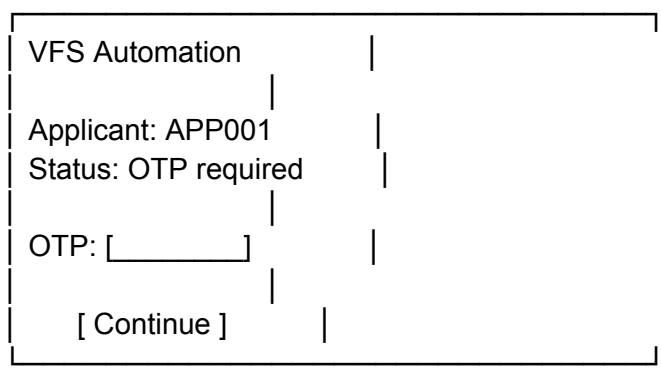
---

12. OTP should also be human-controlled

Same concept:

VFS sends OTP
↓
Automation pauses
↓
User enters OTP
↓
Automation validates page state
↓
Continue

You could build a small local UI:



This is much cleaner than trying to intercept someone's email/SMS automatically.

---

13. Payment should be another hard checkpoint

I recommend:

Automation selects appointment
↓
Shows payment page
↓
PAUSE
↓
Human verifies:
centre
date
time

applicant
fee

↓

Human completes payment

↓

Automation detects confirmation

This gives you a final safety barrier against booking the wrong appointment.

---

14. Add a "booking confirmation guard"

Before the final appointment submission, display:

```
╔═══════════════════════════════════════════╗
║      FINAL BOOKING REVIEW                 ║
╠═══════════════════════════════════════════╣
║ Applicant:      XXXXX XXXXX               ║
║ Passport:       XXXXXXXX                  ║
║ Visa:           Tourist                   ║
║ Centre:         Delhi                     ║
║ Date:           15 Oct 2026               ║
║ Time:           10:30 AM                  ║
╚═══════════════════════════════════════════╝

     [ CANCEL ]    [ CONFIRM ]
```

This single step can prevent expensive mistakes.

---

15. Build logging from day one

Every automation action should generate a log.

Example:

```
2026-09-10 11:02:01  INFO  Browser started
2026-09-10 11:02:04  INFO  Login page detected
2026-09-10 11:02:17  INFO  Login successful
2026-09-10 11:02:19  INFO  Dashboard detected
```

2026-09-10 11:02:24 INFO Application page opened
2026-09-10 11:02:31 INFO Applicant data loaded
2026-09-10 11:02:39 INFO Form validation successful
2026-09-10 11:02:45 INFO Appointment page opened
2026-09-10 11:02:46 INFO No appointment available

For errors:

ERROR
URL:
PAGE:
STATE:
ACTION:
EXCEPTION:
SCREENSHOT:

Save screenshots automatically when something unexpected happens.

---

16. Use Playwright tracing

This is extremely useful when the VFS website changes.

Enable tracing during development:

```
context = browser.new_context()

context.tracing.start(
    screenshots=True,
    snapshots=True,
    sources=True
)
```

At the end:

```
context.tracing.stop(
    path="trace.zip"
)
```

Then you can inspect exactly what happened.

---

17. Build a recovery system

Real-world browser automation fails.

Internet disconnects.

VFS changes a selector.

Session expires.

The site returns an error.

A page takes 30 seconds.

A popup appears.

Your code needs to recover.

Use:

```
Action
 ↓
Success?
 ├── YES → continue
 │
 └── NO
     ↓
  retry?
     ├── YES → retry
     └── NO → screenshot + log + human intervention
```

Don't use unlimited retries.

For example:

MAX_RETRIES = 3

---

18. Keep sessions isolated

For each applicant:

```
Applicant A
 └── Browser Context A
```

Applicant B
└── Browser Context B

Applicant C
└── Browser Context C

Don't mix applicant cookies/session storage.

A persistent context can be useful:

```
context = browser.new_context(
    storage_state="session.json"
)
```

But handle session data securely because it can contain authentication information.

---

19. Don't attempt to defeat VFS's security mechanisms

I can help you build automation around the website, but I would **not** recommend implementing:

- CAPTCHA bypass
- anti-bot fingerprint spoofing
- stealing/reusing another user's session
- bypassing queues
- forged appointment requests
- manipulating backend/API requests
- defeating identity verification
- bypassing rate limits
- purchasing/reserving slots through unauthorized endpoints

Apart from the technical risks, VFS explicitly says its website must not be copied, hacked, misused or abused, and VFS warns about third parties claiming to provide appointment access.

The safe architecture is:

automate the repetitive work; human handles security-sensitive decisions/challenges.

---

20. A very good MVP

If I were building this project with you, I'd make **Version 1** only do this:

START

Load applicant JSON

Validate applicant

Launch Chromium

Open VFS

User logs in

Automation navigates

Fill application

Validate entered values

Navigate appointment page

Read available slots

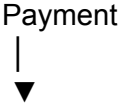
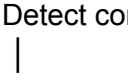
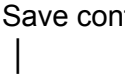
Apply user's appointment rules

Display best matching slot


Human confirms

Proceed to VFS's normal booking flow

CAPTCHA / OTP → human

Payment → human

Detect confirmation

Save confirmation screenshot/reference

END

That is a realistic system rather than a fragile "1000 clicks per second" bot.

---

21. Suggested technology stack

Core

Component	Recommendation
Language	Python
Browser	Chromium
Automation	Playwright
Data validation	Pydantic
Database	SQLite initially
Production DB	PostgreSQL
API	FastAPI
UI	Streamlit initially / React later

Logging	Python logging
Testing	pytest
Screenshots	Playwright
Tracing	Playwright Trace Viewer
Deployment	Docker

---

22. Eventually build a dashboard

Once the basic automation works:

VFS AUTOMATION

Applicants

APP001 John Doe Form Ready
APP002 Jane Doe Appointment Found
APP003 Alex Smith CAPTCHA Required
APP004 David Kumar Completed

Appointment Monitor

Centre Status Next Check
Delhi No slots 12:04
Mumbai Available 12:04
Chennai No slots 12:05

Activity

11:58 Login successful
11:59 Form completed
12:00 Appointment page
12:01 No slot
12:02 No slot

12:03 Appointment found

This is where the project becomes a useful **application**, rather than just a script.

---

23. The development roadmap I'd use

Stage 1

Browser proof-of-concept

Get Playwright opening the correct VFS page.

Stage 2

Navigation

Implement reliable detection of login/dashboard/application/appointment pages.

Stage 3

Applicant model

Build JSON/Pydantic data structure.

Stage 4

Form filling

Automate every non-security-sensitive field.

Stage 5

Validation

Verify every filled field before proceeding.

Stage 6

Appointment reader

Read available centres/dates/times without attempting to bypass protections.

Stage 7

Appointment rules

Implement:

centre preference
date range
time range
visa category

Stage 8

Human checkpoints

Implement:

CAPTCHA
OTP
identity checks
payment
final confirmation

Stage 9

Logging + recovery

Add:

screenshots
trace files
error logs
retry logic
session recovery

Stage 10

Dashboard

Add multiple applicants and appointment monitoring.

Stage 11

Production hardening

Add:

database

encrypted secrets
audit logs
Docker
monitoring
backup
access control

One important point about VFS specifically

The exact workflow is **not universal across VFS**. It changes depending on:

destination country + application type + applicant location/jurisdiction + VFS route.

For example, current VFS pages show different booking rules across routes, including first-come-first-served processes on some routes and random slot releases on others.

So I would **not write the automation against "VFS Global" generically**. I'd first model one exact journey, such as:

India
↓
VFS Delhi
↓
Italy
↓
Tourist / Schengen
↓
Appointment booking

and make that workflow rock-solid before making the framework configurable.

If you give me the **exact VFS route (country + visa type + city/centre)**, I can take this one level deeper and lay out the **actual Playwright project structure, state machine, selectors strategy, database schema, and starter Python code** for that specific workflow.

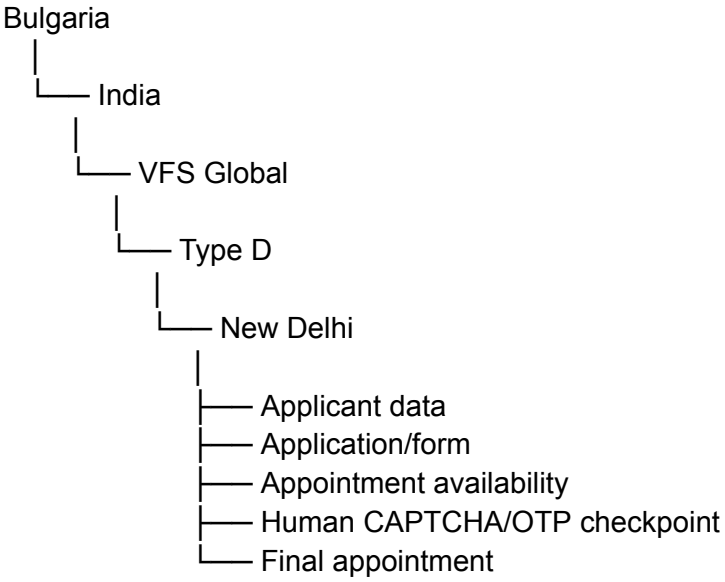
Country: Bulgaria, Visa type: D visa, City/Centre: New Delhi

Perfect. I checked the **current Bulgaria/India VFS and Bulgarian MFA information**, and this changes the architecture somewhat.

For **Bulgaria → India → Type D → New Delhi**, VFS is currently an official partner of the Bulgarian Embassy, and the Bulgarian MFA says that since **1 November 2025**, VFS centres in India can accept **Type D long-stay applications**.

Also, VFS's Bulgaria India homepage currently exposes an **Earliest Available Appointment Date (EAD)** indicator, but explicitly says it is only indicative and that actual availability is reflected after login.

So I would build your automation specifically around this workflow:



The most important design decision

I would **not** make this a "fully automatic VFS bot."

Instead, build a **semi-automated booking assistant**:

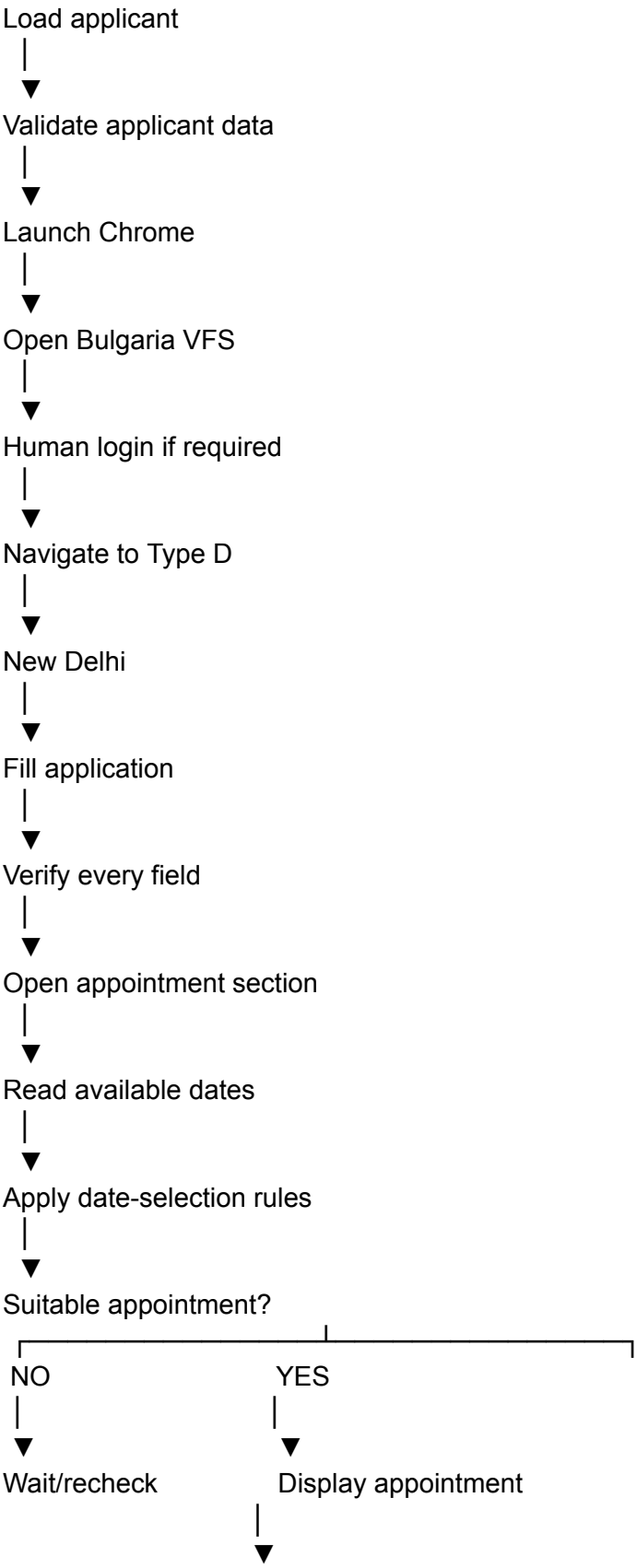
> **Automation does everything repetitive; a human handles CAPTCHA, OTP, payment/security challenges and final confirmation.**

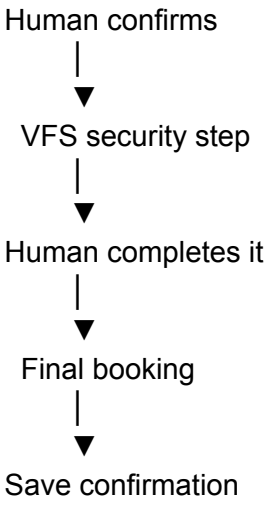
That is both much more robust technically and avoids designing around mechanisms intended to prevent automated abuse.

---

1. Your exact MVP

For your Bulgaria Type-D / New Delhi workflow, I'd target this first:


Load applicant
Validate applicant data
Launch Chrome
Open Bulgaria VFS
Human login if required
Navigate to Type D
New Delhi
Fill application
Verify every field
Open appointment section
Read available dates
Apply date-selection rules
Suitable appointment?
NO
YES
Wait/recheck
Display appointment



---

2. One interesting thing about the current VFS site

The Bulgaria VFS homepage currently has:

> **Find the Earliest Available Appointment Date**

and says the result is dynamic and **doesn't guarantee that the appointment will still be available when you actually try to block it**.

This means your program should distinguish between:

EAD

"VFS says earliest date = X"

and:

Actual slot

"Logged-in booking interface currently has
date X / time Y available."

Your automation should trust the **actual logged-in appointment interface**, not merely the public EAD value.

---

3. I would build the system around a state machine

This is the key to making it reliable.

```
class VFSState:
    START = "start"
    LOGIN = "login"
    DASHBOARD = "dashboard"
    APPLICATION = "application"
    FORM = "form"
    APPOINTMENT = "appointment"
    CAPTCHA = "captcha"
    OTP = "otp"
    REVIEW = "review"
    PAYMENT = "payment"
    CONFIRMATION = "confirmation"
    ERROR = "error"
```

Then:

```
while True:

    state = detect_state(page)

    if state == VFSState.LOGIN:
        handle_login()

    elif state == VFSState.DASHBOARD:
        navigate_to_application()

    elif state == VFSState.APPLICATION:
        select_type_d()

    elif state == VFSState.FORM:
        fill_form()

    elif state == VFSState.APPOINTMENT:
        check_appointments()

    elif state == VFSState.CAPTCHA:
        wait_for_human()
```

```
    elif state == VFSState.OTP:
        wait_for_human_otp()

    elif state == VFSState.REVIEW:
        human_confirmation()

    elif state == VFSState.CONFIRMATION:
        save_confirmation()
        break
```

That's vastly better than:

```
click()
sleep()
click()
sleep()
click()
sleep()
```

---

4. Applicant database

I'd use SQLite initially.

Example:

```
applicant
──────────────────────────────────────
id
first_name
middle_name
last_name
date_of_birth
place_of_birth
nationality
passport_number
passport_issue_date
passport_expiry_date
email
phone
address
visa_type
purpose
```

travel_date
vfs_centre
status
created_at
updated_at

And a separate appointment table:

appointment
_____

id
applicant_id
centre
date
time
status
detected_at
selected_at
confirmation_reference

This lets you later support multiple applicants.

---

5. Don't store applicant data in Python

Bad:

first_name = "John"
passport = "A1234567"

Better:

database
  ↓
Pydantic model
  ↓
automation

For example:

from pydantic import BaseModel
from datetime import date

```
class Applicant(BaseModel):
    first_name: str
    last_name: str
    date_of_birth: date
    passport_number: str
    passport_expiry: date
    nationality: str
    email: str
    phone: str
    visa_type: str
    centre: str
```

Then:

```
applicant = Applicant(...)
```

---

6. Build a form mapping layer

This will become extremely useful because VFS forms can change.

Instead of putting selectors throughout your code:

```
page.locator("#something").fill(...)
```

create:

```
FIELDS = {
    "first_name": {
        "selector": "...",
        "type": "text"
    },

    "last_name": {
        "selector": "...",
        "type": "text"
    },

    "date_of_birth": {
        "selector": "...",
        "type": "date"
    }
}
```

Then:

```
for field, config in FIELDS.items():

    value = getattr(applicant, field)

    page.locator(
        config["selector"]
    ).fill(str(value))
```

When VFS changes one selector, you update the mapping rather than rewriting the application.

---

7. Add field verification

This is **critical for visa applications**.

After filling:

```
locator.fill(value)
```

don't immediately continue.

Verify:

```
actual = locator.input_value()

if actual != value:
    raise FormValidationError(
        f"{field} was not entered correctly"
    )
```

For dropdowns:

```
await expect(
    page.locator(selector)
).to_have_value(expected)
```

For checkboxes:

```
assert await checkbox.is_checked()
```

---

8. Special handling for Type D

Type D is not just another tourist appointment.

The Bulgarian MFA specifically states that VFS India can now accept **long-stay Type D applications**, and its 2025 announcement explains that VFS can collect biometric data for Type D applications. It also describes online consular interviews through specially equipped VFS premises.

So your automation should model Type D separately:

```
Visa
 ├── C
 └── D
     ├── Applicant data
     ├── Purpose/category
     ├── Documents
     ├── Appointment
     ├── Biometrics
     └── Possible consular interview
```

Don't assume a Schengen-style workflow applies.

---

9. Appointment engine

I'd make this a separate module:

```
appointment/
   detector.py
   parser.py
   rules.py
   selector.py
```

Detector

Determines:

```
Is appointment page loaded?
Is centre selected?
Is calendar available?
Is slot available?
```

Is there an error?

Parser

Converts the website into:

```
[
   {
      "date": "2026-10-12",
      "time": "09:30",
      "centre": "New Delhi"
   },
   {
      "date": "2026-10-13",
      "time": "10:00",
      "centre": "New Delhi"
   }
]
```

Rules

```
def acceptable(slot):

   if slot["centre"] != "New Delhi":
      return False

   if slot["date"] < earliest_date:
      return False

   if slot["date"] > latest_date:
      return False

   return True
```

Selector

```
valid_slots = [
   slot for slot in slots
   if acceptable(slot)
]

best = sorted(
   valid_slots,
   key=lambda x: (
      x["date"],
      x["time"]
```

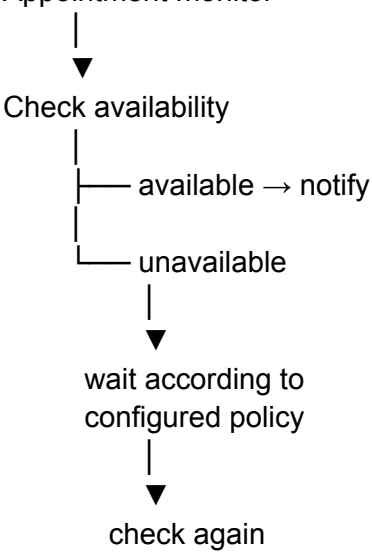
```
    )
)[0]
```

---

10. Don't blindly refresh the appointment page

This is especially important.

I'd build:



with limits such as:

```
monitor:
  enabled: true
  maximum_session_minutes: 60
  maximum_checks: 30
  retry_on_network_error: 3
```

If VFS responds with a rate limit, security warning, or similar restriction:

STOP

Don't try to circumvent it.

---

11. Notification system

This is one area I'd automate aggressively.

When a matching appointment appears:

New Delhi
Type D
15 October 2026
10:30 AM

send:

🔔 Appointment Found

Visa: Bulgaria Type D
Centre: New Delhi
Date: 15 Oct 2026
Time: 10:30 AM

[Open Browser]
[Confirm]
[Reject]

You could eventually use:

- Telegram
- email
- desktop notification
- WhatsApp via an appropriate authorized integration

For the MVP, email/desktop notification is enough.

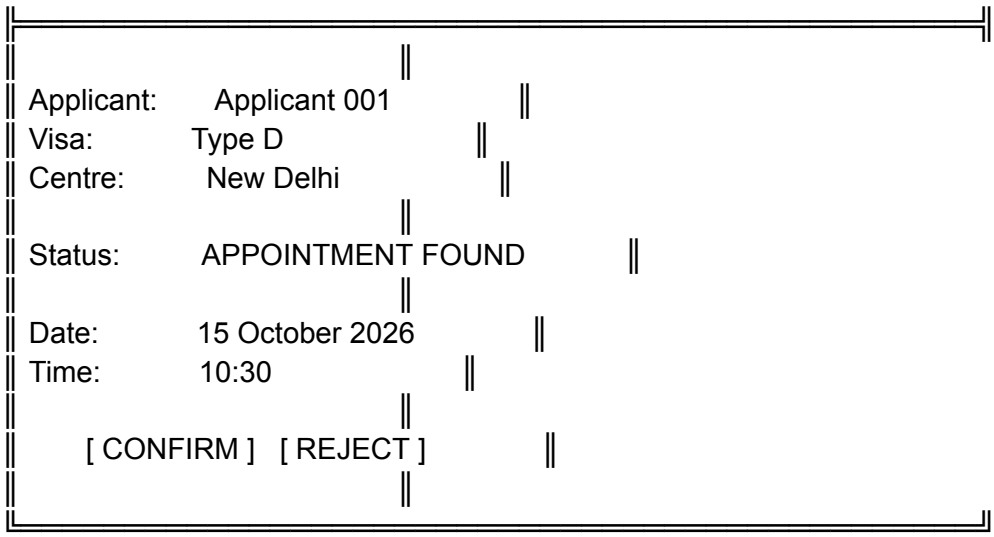
---

12. Human checkpoint UI

I'd actually create a tiny local dashboard.

Something like:

```
╔═══════════════════════════════════════╗
║      BULGARIA VISA AUTOMATION         ║
```



The browser remains visible behind it.

---

13. CAPTCHA handling

Don't attempt to automate around the CAPTCHA.

Your code should detect the situation:

```
if captcha_detected(page):
    status = "CAPTCHA_REQUIRED"

    notify_user(
        "CAPTCHA requires human action"
    )

    wait_until_cleared(page)
```

Then the user interacts with the actual browser.

The automation resumes when the expected page state changes.

---

14. OTP handling

Same approach.

VFS
↓
OTP requested
↓
automation pauses
↓
user enters OTP
↓
automation detects successful verification
↓
continue

Don't put SMS/email passwords or OTP extraction into the automation unnecessarily.

---

15. Final booking safety

Before committing the appointment, show:

═══════ FINAL REVIEW ═══════

Country: Bulgaria
Visa: Type D
Centre: New Delhi

Applicant:
XXXXXXXX XXXXXXXX

Appointment:
15 October 2026
10:30 AM

═══════════════════════════════

[CANCEL]

[CONFIRM BOOKING]

I strongly recommend this even if everything else is automatic.

---

16. Project structure I'd use

For your exact project:

```
bulgaria-vfs/
│
├── main.py
│
├── config/
│   ├── settings.yaml
│   └── appointment_rules.yaml
│
├── database/
│   ├── db.py
│   └── models.py
│
├── vfs/
│   ├── browser.py
│   ├── state.py
│   ├── login.py
│   ├── navigation.py
│   ├── application.py
│   ├── form.py
│   ├── appointment.py
│   ├── captcha.py
│   ├── otp.py
│   └── confirmation.py
│
├── applicant/
│   ├── model.py
│   ├── validator.py
│   └── repository.py
│
├── notifications/
│   ├── email.py
│   └── desktop.py
│
├── ui/
│   └── dashboard.py
│
├── logs/
│
├── screenshots/
```

```
│
├── traces/
│
└── tests/
```

---

17. Recommended development order

Don't try to write the whole thing at once.

Week/Stage 1

Build:

Playwright
↓
Open Bulgaria VFS
↓
Detect page

Stage 2

Build:

Login
↓
Dashboard
↓
Type D
↓
New Delhi

Stage 3

Build:

Applicant JSON
↓
Form filling
↓
Field verification

Stage 4

Build:

Appointment page
↓
Calendar detection
↓
Slot extraction

Stage 5

Build:

Date preferences
↓
Slot ranking
↓
Notification

Stage 6

Build:

CAPTCHA pause
↓
OTP pause
↓
Human confirmation

Stage 7

Build:

Database
↓
Dashboard
↓
Logs
↓
Error recovery

One thing I would do before writing the actual selectors

Because the VFS page is dynamic, I don't want to invent selectors for your particular Type-D workflow.

The next useful step is to inspect the actual Bulgaria VFS pages you see.

If you can provide **screenshots of the VFS Type-D application/appointment pages** (with passport numbers, emails, phone numbers, OTPs, passwords and other sensitive information hidden), I can map the UI into a concrete Playwright workflow.

Upload a photo

Then I can give you the next layer:

1. Exact page/state map
2. Selector strategy
3. Playwright Python classes
4. Applicant JSON schema
5. SQLite schema
6. Appointment detection algorithm
7. CAPTCHA/OTP human checkpoint
8. Error/retry architecture
9. Complete MVP project skeleton
10. First working Python implementation

The **official Bulgaria VFS portal** is here: Bulgaria VFS Global — India. The Bulgarian MFA also confirms New Delhi as one of the Indian VFS locations accepting Bulgarian visa applications, including the newly enabled Type-D process.